

The top of the image features a decorative header with a dark red background. It contains several overlapping semi-circular shapes. Some of these shapes are filled with a pattern of concentric dotted lines, while others are solid dark red. The patterns are arranged in a way that creates a sense of depth and movement.

Smart Restaurant Ordering System

Problem Statment

- Manual order taking may cause errors and delays.
- Difficult to track customer orders and order status.
- Manual billing can lead to calculation mistakes.
- Restaurant staff have increased workload.
- Records may become scattered and difficult to manage.
- A centralized digital system is needed for efficient restaurant operations

•.

Objectives

- Digitize the restaurant ordering process.
- Reduce manual work and ordering errors.
- Allow customers to view the menu and place orders easily.
- Automate bill calculation based on selected items.
- Maintain organized customer, menu, order, and billing records.
- Enable staff to manage menu items and update order status.
- Improve order processing speed and overall restaurant efficiency.

Proposed System

- The proposed Smart Restaurant Ordering System provides a centralized platform for managing restaurant orders and related information. Customers can browse the menu, select food items, specify quantities, and place orders. The system stores customer, menu, order, and billing information in a structured database. Restaurant staff can manage menu items, view customer orders, update order status, and generate bills.

System Architecture

The system follows a **three-layer architecture** consisting of the Frontend, Backend, and Database layers. The frontend provides the user interface for customers and restaurant staff. The backend handles application logic, order processing, billing, and communication with the database. The MySQL database stores customer, menu, order, and billing information. JDBC is used to connect the Java backend with the MySQL database.

- Customer / Staff



- Frontend

- HTML + CSS + JavaScript



- Backend

- Java



- JDBC



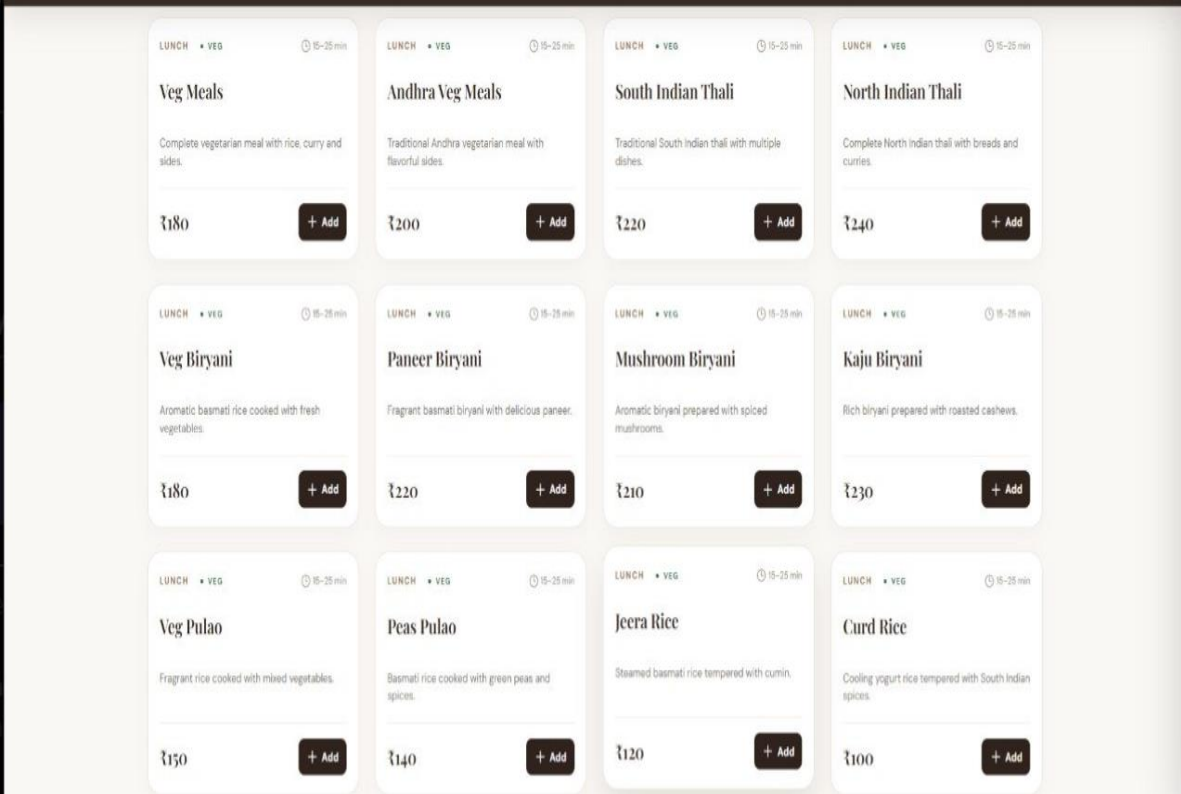
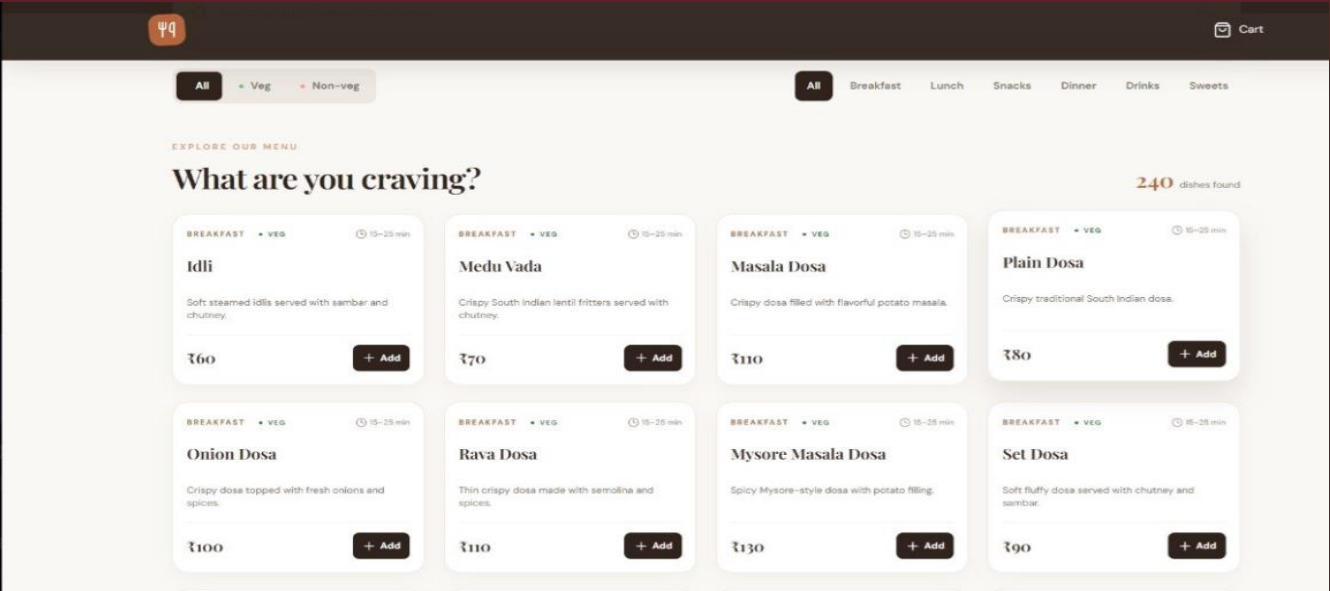
- Database

- MySQL

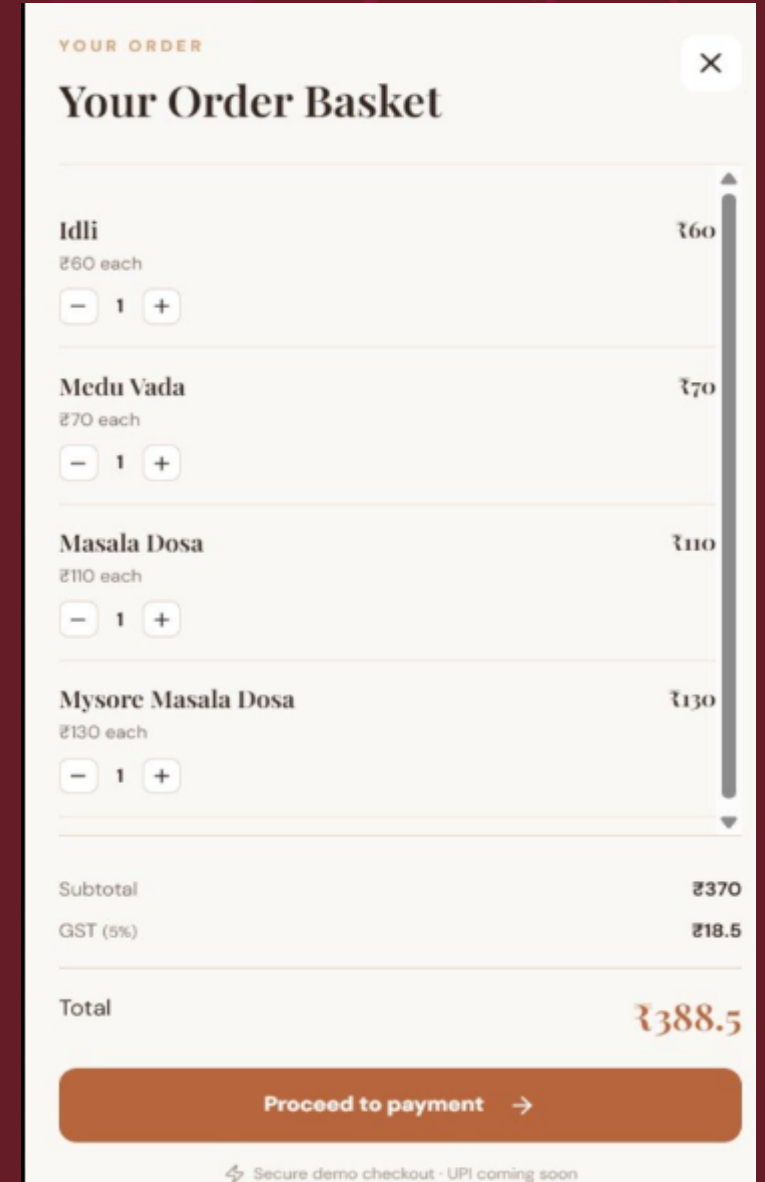
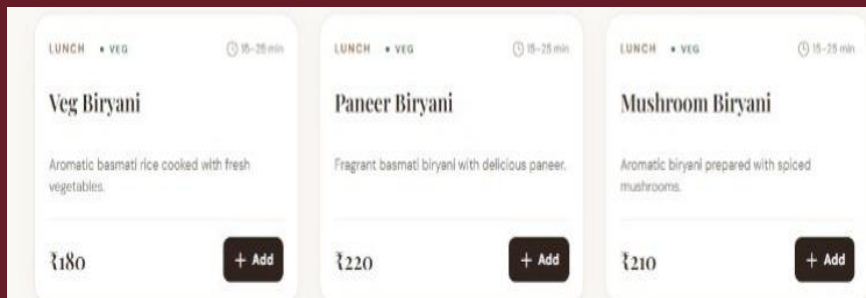
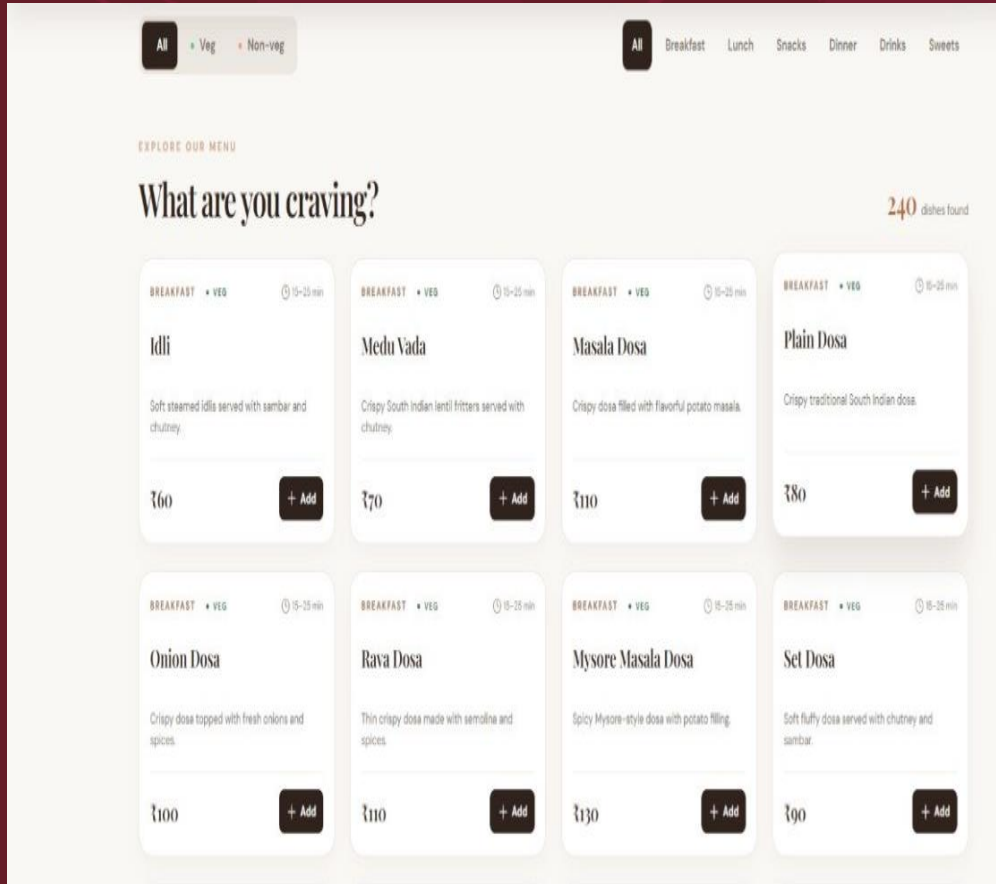
Modules / Functionalities

- Customer Module:** View menu, select items, choose quantity, and place orders.
- Menu Management:** Add, update, delete, and manage food item details.
- Order Management:** Store customer orders, view order details, and update order status.
- Billing Module:** Calculate the total amount and display the final bill.
- Staff/Admin Module:** Manage menu items, customer orders, and billing records.
- Database Module:** Store customer, menu, order, and billing information securely.

Frontend Design



Frontend Implementation - Screenshots



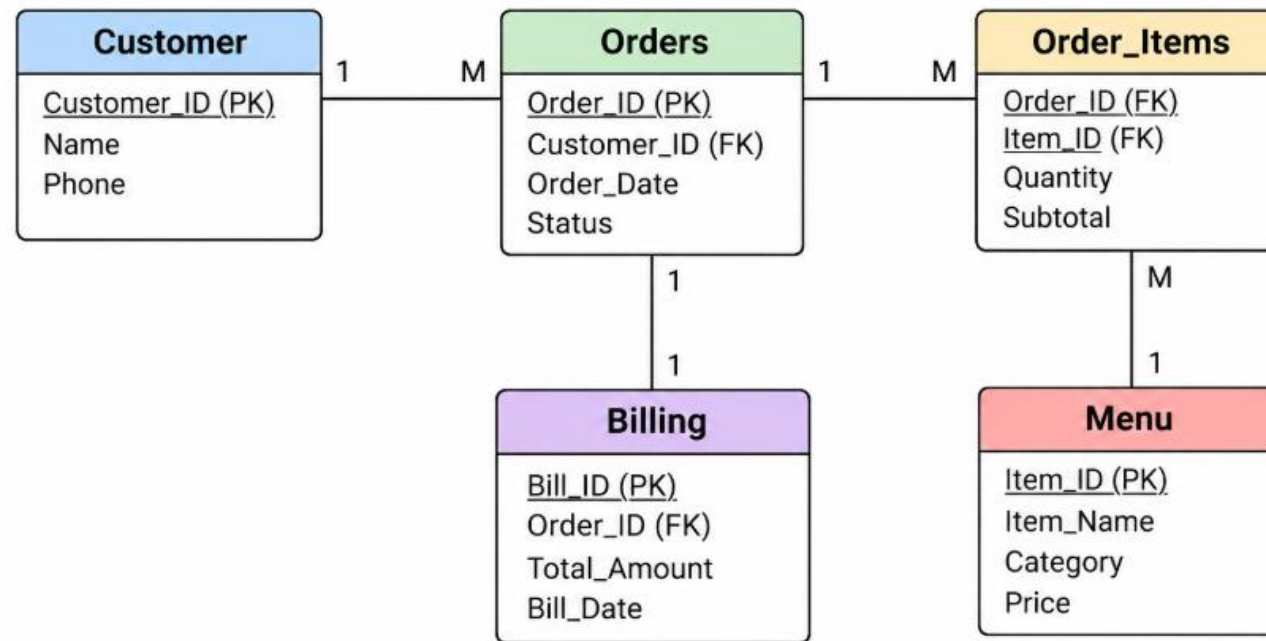
Database Design

```
1 CREATE DATABASE viraham_restaurant;
2
3 USE viraham_restaurant;
4
5 USE viraham_restaurant;
6
7 -- 1. USERS
8 CREATE TABLE users (
9     user_id INT AUTO INCREMENT PRIMARY KEY,
10     name VARCHAR(100) NOT NULL,
11     email VARCHAR(150) NOT NULL UNIQUE,
12     password_hash VARCHAR(255) NOT NULL,
13     role ENUM('OWNER', 'STAFF') NOT NULL,
14     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
15 );
16
17 -- 2. RESTAURANT TABLES
18 CREATE TABLE restaurant_tables (
19     table_id INT AUTO INCREMENT PRIMARY KEY,
20     table_number INT NOT NULL UNIQUE,
21     qr_code VARCHAR(255) UNIQUE,
22     status ENUM('AVAILABLE', 'OCCUPIED') DEFAULT 'AVAILABLE',
23     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
24 );
25
26 -- 3. MENU CATEGORIES
27 CREATE TABLE menu_categories (
28     category_id INT AUTO INCREMENT PRIMARY KEY,
29     category_name VARCHAR(100) NOT NULL UNIQUE,
30     description VARCHAR(255),
31     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
32 );
33
34 -- 4. MENU ITEMS
35 CREATE TABLE menu_items (
36     item_id INT AUTO INCREMENT PRIMARY KEY,
```

```
365 USE viraham_restaurant;
366
367 INSERT INTO menu_items
368 (category_id, food_type, meal_type, item_name, description, price, image_url, is_available)
369 VALUES
370 (2, 'NON_VEG', 'LUNCH', 'Chicken Biryani', 'Aromatic basmati rice with tender chicken', 260.00, NULL, TRUE),
371 (2, 'NON_VEG', 'LUNCH', 'Mutton Biryani', 'Rich basmati biryani with tender mutton', 340.00, NULL, TRUE),
372 (2, 'NON_VEG', 'LUNCH', 'Egg Biryani', 'Fragrant biryani with boiled eggs', 190.00, NULL, TRUE),
373 (2, 'NON_VEG', 'LUNCH', 'Chicken Fry Biryani', 'Biryani topped with crispy chicken fry', 290.00, NULL, TRUE),
374 (2, 'NON_VEG', 'LUNCH', 'Chicken Dum Biryani', 'Slow cooked dum biryani with chicken', 280.00, NULL, TRUE),
375 (2, 'NON_VEG', 'LUNCH', 'Mutton Dum Biryani', 'Slow cooked dum biryani with mutton', 360.00, NULL, TRUE),
376 (2, 'NON_VEG', 'LUNCH', 'Chicken Pulao', 'Fragrant rice cooked with chicken', 220.00, NULL, TRUE),
377 (2, 'NON_VEG', 'LUNCH', 'Mutton Pulao', 'Fragrant rice cooked with mutton', 300.00, NULL, TRUE),
378 (2, 'NON_VEG', 'LUNCH', 'Chicken Fried Rice', 'Indo-Chinese fried rice with chicken', 200.00, NULL, TRUE),
379 (2, 'NON_VEG', 'LUNCH', 'Egg Fried Rice', 'Fried rice with scrambled egg', 170.00, NULL, TRUE),
380 (2, 'NON_VEG', 'LUNCH', 'Chicken Schezwan Rice', 'Spicy Schezwan fried rice with chicken', 220.00, NULL, TRUE),
381 (2, 'NON_VEG', 'LUNCH', 'Chicken Meals', 'Complete meal with chicken curry, rice and sides', 260.00, NULL, TRUE),
382 (2, 'NON_VEG', 'LUNCH', 'Andhra Chicken Meals', 'Traditional Andhra chicken meal', 280.00, NULL, TRUE),
383 (2, 'NON_VEG', 'LUNCH', 'Mutton Meals', 'Complete meal with mutton curry and rice', 340.00, NULL, TRUE),
384 (2, 'NON_VEG', 'LUNCH', 'Chicken Curry Rice', 'Chicken curry served with steamed rice', 230.00, NULL, TRUE),
385 (2, 'NON_VEG', 'LUNCH', 'Mutton Curry Rice', 'Mutton curry served with steamed rice', 300.00, NULL, TRUE),
386 (2, 'NON_VEG', 'LUNCH', 'Fish Curry Rice', 'Fish curry served with rice', 240.00, NULL, TRUE),
387 (2, 'NON_VEG', 'LUNCH', 'Prawn Curry Rice', 'Prawn curry served with rice', 280.00, NULL, TRUE),
388 (2, 'NON_VEG', 'LUNCH', 'Chicken 65 Rice Bowl', 'Rice bowl topped with Chicken 65', 230.00, NULL, TRUE),
389 (2, 'NON_VEG', 'LUNCH', 'Egg Curry Rice', 'Egg curry served with steamed rice', 180.00, NULL, TRUE);
390
391 USE viraham_restaurant;
392
393 INSERT INTO menu_items
394 (category_id, food_type, meal_type, item_name, description, price, image_url, is_available)
395 VALUES
396 (1, 'NON_VEG', 'SNACKS', 'Chicken 65', 'Crispy spicy fried chicken pieces', 180.00, NULL, TRUE),
397 (1, 'NON_VEG', 'SNACKS', 'Chicken 555', 'Spicy Andhra-style crispy chicken', 190.00, NULL, TRUE),
398 (1, 'NON_VEG', 'SNACKS', 'Chicken Pakora', 'Crispy chicken pieces coated in spiced batter', 180.00, NULL, TRUE),
399 (1, 'NON_VEG', 'SNACKS', 'Chicken Majestic', 'Tender chicken strips with creamy spicy seasoning', 200.00, NULL, TRUE),
400 (1, 'NON_VEG', 'SNACKS', 'Chicken Lollipop', 'Fried chicken wings served with spicy sauce', 220.00, NULL, TRUE),
401 (1, 'NON_VEG', 'SNACKS', 'Chicken Wings', 'Crispy fried chicken wings', 210.00, NULL, TRUE),
402 (1, 'NON_VEG', 'SNACKS', 'Chicken Tikka', 'Grilled marinated chicken pieces', 240.00, NULL, TRUE),
403 (1, 'NON_VEG', 'SNACKS', 'Chicken Seekh Kebab', 'Minced chicken kebabs grilled with spices', 230.00, NULL, TRUE),
404 (1, 'NON_VEG', 'SNACKS', 'Mutton Seekh Kebab', 'Spiced minced mutton grilled on skewers', 280.00, NULL, TRUE),
405 (1, 'NON_VEG', 'SNACKS', 'Mutton Shami Kebab', 'Soft minced mutton kebabs with aromatic spices', 260.00, NULL, TRUE),
406 (1, 'NON_VEG', 'SNACKS', 'Fish Fry', 'Crispy shallow-fried marinated fish', 220.00, NULL, TRUE),
```

ER Diagram

ER Diagram



Database Tables / Schema

Database Design — Main Tables

Table	Key Fields (examples)
Customer	customer_id (PK), name, phone, email
Menu	menu_id (PK), item_name, category, price, availability
Orders	order_id (PK), customer_id (FK), order_date, order_status, total_amount
Order_Items	order_item_id (PK), order_id (FK), menu_id (FK), quantity, subtotal
Billing	bill_id (PK), order_id (FK), total_amount, payment_status

Notes

- PK = Primary Key; FK = Foreign Key
- Use indexes on customer_id, menu_id, order_date for performance
- Constraints: NOT NULL, CHECK (price & quantity ≥ 0), UNIQUE (email optional)

Sample Database Records

Customer Table		
Customer_ID	Name	Phone
C001	John Smith	9876543210
C002	Priya Sharma	9123456780
C003	Rahul Mehta	9988776655
C004	Sneha Patel	9012345678

Menu Table			
Item_ID	Item_Name	Category	Price
M001	Margherita Pizza	Main Course	250
M002	Veg Burger	Fast Food	150
M003	Paneer Tikka	Starter	220
M004	Chocolate Brownie	Dessert	120

Orders Table			
Order_ID	Customer_ID	Order_Date	Status
O001	C001	2025-09-10	Completed
O002	C002	2025-09-12	Preparing
O003	C003	2025-09-15	Pending
O004	C004	2025-09-16	Completed

Order_Items Table			
Order_Item_ID	Order_ID	Item_ID	Quantity
OI001	O001	M001	2
OI002	O001	M003	1
OI003	O002	M002	1
OI004	O003	M001	1
OI005	O004	M004	2

Billing Table			
Bill_ID	Order_ID	Total_Amount	Payment_Status
B001	O001	720	Paid
B002	O002	150	Pending
B003	O003	250	Paid
B004	O004	240	Pending

These sample records show how the database stores customer details, menu items, orders, and billing information in the restaurant management system.

Current Progress

- Frontend design and restaurant menu interface are completed.
- Menu browsing, item selection, cart, and order summary are working.
- Basic billing/total calculation is implemented.
- Database structure and ER diagram have been designed.
- Java backend, JDBC connectivity, and MySQL integration are currently in progress.**
- Final testing and complete database integration are pending.

Challenges / Issues Faced

- Setting up the project environment and required dependencies.
- Integrating the frontend with the Java backend.
- Establishing JDBC connectivity with MySQL.
- Designing proper database tables and relationships.
- Handling order, cart, and billing data correctly.
- Maintaining data consistency while updating order status.
- Testing the complete flow from menu selection to billing.

Future Work

- Complete Java backend development.
- Integrate MySQL database using JDBC.
- Add user login and registration.
- Develop admin/staff dashboard.
- Improve order tracking and status updates.
- Add online payment functionality.
- Perform complete testing and improve security.
- Make the system more responsive and user-friendly

Conclusion

- The **Smart Restaurant Ordering System** helps simplify restaurant operations by reducing manual work and organizing menu, order, customer, and billing information in a structured way. The system is designed to improve order accuracy, reduce delays, and make restaurant data easier to manage. Further development will focus on completing the backend, database integration, and additional features such as login, order tracking, and online payments.